Towards a BRS-based Model for IoT Hybrid Fault Tolerance

Sahar Smaali*, Rachida Boucebsi†, Wafa Ghemmaz‡
LIRE Laboratory
Constantine 2- Abdelhamid Mehri University
Constantine, Algeria
Email: *sahar.smaali@univ-constantine2.dz, †rachida.boucebsi@univ-constantine2.dz, ‡wafa.ghemmaz@univ-constantine2.dz

2022 International Conference on Advanced Aspects of Software Engineering (ICAASE) | 978-1-6654-9202-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICAASE56196.2022.9931572

***Abstract*—In an increasingly connected world, the Internet of Things (IoT) has emerged as a framework of technologies and paradigms for the future of the Internet. However, it suffers from failures that can have undesirable consequences or even cause fatal accidents, especially in critical domains. Therefore, Fault Tolerance is a major challenge for assuring IoT systems dependability. In this article, we tackle this problem by proposing a hybrid approach, which consists of a generic microservice architecture baptized H-FaTMA. It enables proactive and reactive fault tolerance by providing detection of things failures as well as continuous and real-time monitoring of their states. In addition, it offers mechanisms to strengthen the reliability of the designed systems. We lean on formal methods to verify the functioning of H-FaTMA. Thus, we adopt Bigraphical Reactive Systems (BRS) to define our formal model that covers the description of architectural elements and simulates fault detection and recovery strategies.**

***Keywords*—IoT, Dependability, Fault tolerance, Microservice Architecture, Formal methods, Bigraphical reactive systems.**

I. Introduction

IoT (Internet of Things) is a global network of smart devices where the degree of heterogeneity, dynamicity and complexity between these objects is rapidly growing. Consequently, ensuring systems dependability becomes a major IoT challenge. Indeed, any failure occurring at any architectural level of any IoT application may lead to overall IoT system failure. Moreover, financial losses and environmental damages may result. For this reason, adopting a fault-tolerance strategy for IoT systems is indispensable.

Fault tolerance (FT) is one of the factor keys to ensure system dependability by allowing the system to continue to deliver a reliable service despite the presence of failures and errors [2]. We may distinguish between two types of FT policies: (1) Reactive policies that minimize the impact of failures on the system when the failure effectively occurs; and (2) Proactive policies that aim at avoiding recovery from failures by predicting errors and replacing the suspected element. In fact, developing a fault-tolerant IoT system is a complicated task especially with the extremely large variety of IoT and edge devices, data computing technologies, networks, and other resources that may be involved in the development process [1].

The FT issue has been more and more studied and attracted more attention especially in the last few years where diverse contributions are proposed. The majority of these contributions focused on the IoT fault tolerance from hardware and network aspects. However, software engineering is not largely addressed even if it provides good support for fault detection and recovery of failed services; it remains largely under-explored in contemporary scientific literature [1].

In [2], the authors defined a generic approach named FaTMA (Fault Tolerance- Microservice Architecture for IoT) where the major aim is to solve dependability issues in IoT systems. Their architecture takes advantage of both the Fog infrastructure as well as the microservices architectural style to deal with the different failures that can occur in the different layers of an IoT architecture. In addition, their approach is based on BRS (Bigraphical Reactive Systems) [3] [4]as a semantic framework to model both the elements of each architecture layer as well as the simulation and analysis of fault detection and management. However, FaTMA allows the discovery of device shutdowns using health messages. Moreover, proactive FT is not supported at all.

In a stream of this work, we propose H-FaTMA (Hybrid FaTMA); an extension of FaTMA; that covers both proactive and reactive FT. The main advantages of our approach are:

- It separates fault detection, data processing, recovery strategies and device management.
- It is based on microservices style and Edge devices to provide decentralized control and fault management, low latency as well as weak coupling between services, and more flexibility.
- A BRS-based formal model is proposed in order to specify H-FaTMA architectural and behavioral aspects.
- It provides a BRS model that allows analysis and simulation of various fault tolerance and system recovery scenarios at the early phases of software life cycle.

The remainder of the paper is structured as follows: In Section 2, related work is discussed. Section 3 presents H-FaTMA architecture and how it supports proactive and reactive FT. Section 4 explains our formal BRS model that permits the simulation of H-FaTMA elements behavior. Section 5 presents an illustrative example. Finally, a conclusion is given in Section 6.

II. Related work

Fault tolerance (FT) is one of the main challenging issues in IoT systems that increasingly attracts more research interest.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:52 UTC from IEEE Xplore. Restrictions apply.

TABLE I
H-FATMA ARCHITECTURAL ELEMENTS

Roles	Microservice	Layer	Provided Interface	Required Interface
Data gathering, processing and storage	Data Controller	Edge	gathered data	contextual data
	Data Processing	Fog	gathered data	processed data, gathered data
	Data Storing	Cloud	data saving, data	
	Application	Cloud	user requests	data
Monitoring and Detection	Things Reactive Monitor	Edge	Faults events	gathered data
	Things Proactive Monitor	Fog	Trained Model	data
	Proactive Model Trainer	Cloud	Faults events	processed data
Recovery and Maintenance	Decision Maker	Edge		Faults events, actions, report
	Things manager	Edge	actions	executing actions
	Fog-Things Supervisor	Fog	report	report
	Cloud-Things Supervisor	Cloud	report	report, actions

In the literature, several approaches have been proposed such as [5], [6], [8] and [7] tackling FT from hardware and network point of view. In this section, we present some of existing approaches tackling FT form a software engineering perspective.

In [9], a decentralized solution is introduced in order to ensure fault tolerance in the WuKong middleware. In this last, FT is supported by two major mechanisms: the replicated services and the decentralized fault recovery. Failed devices are discovered using Heartbeats messages and decentralized protocol to ease the master burden by offloading the responsibility to the network for monitoring devices in the middleware. Therefore, the failure of a device does not disable the entire failure detection system.

In [10], the authors introduce a smart home architecture in which each home has a master controller. In this architecture, the data is collected based on a direct communication between sensors and the local master which can also gather other relevant data from Cloud. By locally extracting knowledge, the sensors are tuned in real time in order to minimize the redundancy in usage and power consumption. However, the output of some sensors can be predicted from others with high accuracy when simulating this architecture on a smaller set.

In [11], a framework based on microservices architecture for reactive and proactive FT in IoT systems is proposed. The architecture is based on two main microservices: Real-Time FT and Predictive FT. The former uses complex event processing to analyze stream data for rapid error recovery. The latter is based on machine learning in order to learn fault patterns and mitigate future faults before they occur. In [12], the same authors proposed Complex Patterns of Failure (CPoF), using Complex Event Processing (CEP) and Machine Learning (ML). Error-detection strategies are defined as non-deterministic finite automata (NFA) and implemented via CEP systems. Reactive-FT support is monitored and learned from to train ML models that proactively handle imminent future occurrences of known errors.

The authors in [13] have proposed a multi-layer fault tolerance approach named FaTEMa. This approach is based on the idea that the adoption of multi-layer fault tolerance can produce more dependable systems by distributing the responsibilities of tolerating faults across multiple layers of an IoT system. A communication standard is defined in order to facilitate interoperability between diverse fault tolerance mechanisms. However, really few works have addressed formal analysis and verification IoT fault tolerance, even if formal methods promote the analysis and verification at the early phases of system life cycle. Our approach is based on BRS as a semantic framework to model, simulate and analyse fault detection and management.

III. H-FATMA: HYBRID FAULT TOLERANCE MSA FOR IOT

In this paper, we propose H-FaTMA, a hybrid fault tolerant microservice architecture for building IoT systems. The choice of this architectural style is justified by its power to reinforce fault tolerance (FT) and resilience; due to the fact that microservices are completely independent and operate in an autonomous way. Therefore, the failure of one or more microservices does not necessarily cause a total system shutdown.

Our architecture is an extension of FaTMA [2] , an architecture designed to detect smart devices failures by providing continuous and real-time monitoring of their states. It offers mechanisms to improve system reliability and resiliency while ensuring fault tolerance in the different layers of an IoT system. FaTMA consists of three layers (Things, Fog and Cloud) where each layer element's monitor the proper functioning of the components in the lower layer. FaTMA recovery technique is based on redundancy which can be hardware (smart devices) or software (Fog and Cloud microservices). In this paper, we focus only on failures that occur at the Things layer.

H-FaTMA reinforced FaTMA multi-layer monitoring and control by introducing the Edge layer (see figure 1). This latter brings the processing and networking services closer to the objects themselves, which reduces fault detection time and offers faster recovery. H-FaTMA gives a clear separation between fault tolerance operations namely monitoring, analysis as well as fault detection, recovery and assessment. This is achieved by assigning specific roles to each microservice (see Table 1). These microservices are distributed over the three layers "Edge, Fog, Cloud" and communicate with each other via messages.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:52 UTC from IEEE Xplore. Restrictions apply.

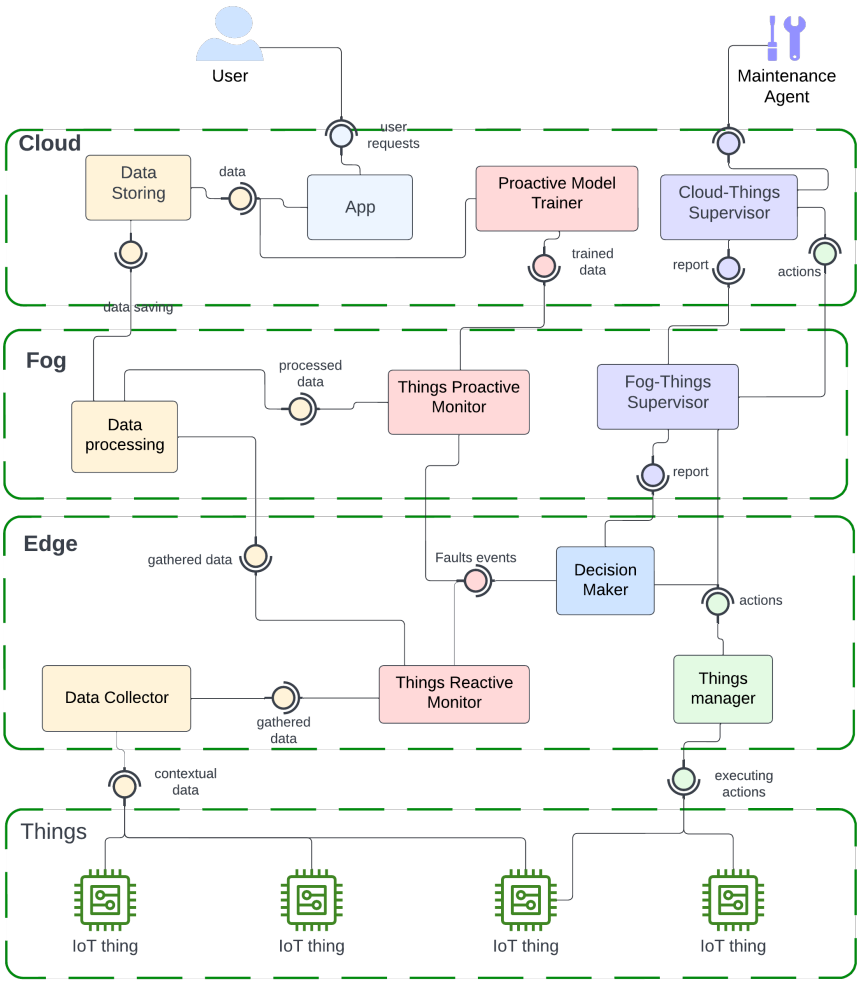


Fig. 1. H-FaTMA Generic Architecture.

In [2], authors have focused on smart devices breaking down while neglecting other types of faults such as sensor deviation. However, H-FaTMA judges a failure in a different way by considering any data inconsistency as a failure. This is done by analyzing the data received by the lower layer. As an example, Things Reactive Monitor considers a failure of any two sensors of the things layer sending incompatible values. A comparison between the received data and the actions of the actuators is also made in order to ensure that the system does not present an anomaly. For instance, if a temperature sensor takes as a value a high temperature, while the air conditioning actuator exerts the cooling action during a time t, we speak about a failure.

In fact, H-FaTMA supports two complementary mechanisms of fault tolerance: (1) reactive FT (which FaTMA already covers) and (2) proactive FT where it discovers an anomaly state even before a device failure.

At the Edge layer, the *Data collector* microservice is responsible for collecting data sent by the sensors (from the Things layer) and transmitting them to the *Things Reactive Monitor* microservice. The latter is responsible for an initial analysis of the data in order to detect the possibility of a hardware/software failure. In fact, Things Reactive Monitor plays the role of a firewall that forbids faulty data from reaching the upper layers and more precisely to the *Data Processing* and *Data Storing* microservices in the fog and cloud respectively.

After scaling and standardization, the processed data will be used for training and fault prediction by the *Things Proactive Monitor* and *Proactive Model Trainer*. This microservice exploits machine and deep learning techniques and algorithms to discover any data anomalies or outliers. The *proactive fault detection* is situated at the Fog layer as it is closer to the things layer compared to the cloud and gives more computational resources that sufficient to run intelligent algorithms compared to the Edge capacities. While, the model training is done in the Cloud by *Proactive Model Trainer* as this task is computationally intensive and has to be optimized over time by analysing new data.

Whether it's reactive or proactive fault detection, the *Decision*

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:52 UTC from IEEE Xplore. Restrictions apply.

Maker microservice chooses the appropriate recovery strategy according to the failure nature and severity (low, normal, high). The recovery strategies such as replication will be to be executed by the *Things Manager* microservice that offers direct control of the Things layer elements. The origin of a failure may be of a single object (isolated failure) or of several objects that stop working simultaneously. In both cases, the recovery is done by treating each failure independently of other failures (isolated). In addition, the *Decision Maker* notifies the maintenance agents of any type of failure that requires their intervention to repair or replace the crashed devices.

IV. H-FaTMA Bigraphical model

In this section, we illustrate the ability of BRS to cover both static and dynamic aspects of the proposed multi-layered fault tolerant architecture. Our BRS based model associate to each architectural elements of H-FaTMA a precise and rigorous semantics (static aspect). In addition, I allows the simulation of fault detection and system recovery to offer hybrid FT (dynamic aspect). At a high level of abstraction, an IoT system can be seen as a set of links and nodes distributed over regions. Elements interaction and cooperation is defined by a set of reaction rules. Thus, we define our BRS-based model as a couple: $Bis = (FT, React)$; where The FT is a Bigraph that models the infrastructure of the system (static aspect) and React is the set of reaction rules that define its behavior (dynamic aspect).

A. Bigraphs presentation

Bigraphical Reactive Systems (BRS) theory was initially introduced by Milner (2008) for modeling distributed mobile code [3] [4]. A BRS is defined by a bigraph that consists of a set of reaction rules allowing to define how a given system may evolve over time. Bigraphs (see Fig. 2) is a combination of two separate graph structures (places and links graphs) modeling both connectivity and locality aspects of a system.

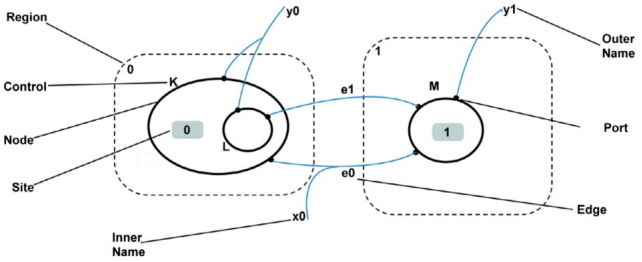


Fig. 2. Bigraph anatomy.

Places graph specifies system entities spreading through a certain nodes hierarchy. While, the links graph captures connections between nodes. It consists of a hypergraph that abstracts the possible entities' interactions using edges (hyper-edges). Each edge (see 2) connects at least two nodes. Places graph consists of a forest of trees, called roots, describing system' parts and able to nest a set of nodes. The nodes (system logical/ physical components) as illustrated in Fig. 2 can nest or/and be nested in other nodes. Each node is characterized by a control; a type identifier combined with an arity to define the number of associated ports. A bigraph may have one or more sites expressing the ability to hold other bigraphs (or regions). A bigraph interacts with other ones (external environment) through inner and/or outer names that figure in the interfaces [3] [4].

Bigraphs can be also described by means of textual term language, allowing to conceive more complex bigraphs thanks to several available operations. On the other hand, a sorting discipline may be added to bigraph definition in order to avoid inadmissible bigraphs construction; imposing some formation rules (constraints or properties). In BRS, a reaction rule R having the following format: $R : B \rightarrow B'$ represents the reconfiguration of a bigraph B to B' in some bigraphical context.

In the following subsections, we detail each part of our proposed model.

B. Bigraphical Model for H-FaTMA Structure

According to our formalisation approach, we define the H-FaTMA structure as a bigraph FT having four independent regions (see Fig. 3), representing $Things, Edge, Fog$ and $cloud$ layers respectively. *Thing* Region holds a set of smart devices which are divided into two categories represented by the *Main* and *Backup* nodes. The *Main* node groups together all the primary objects (specified by the $Thing$ control), while the *Backup* node groups together the backup objects that will be exploited in case of a primary object failure. *Edge, Fog* and *Cloud* Regions represent the Edge, Fog and cloud layers where each microservice of these layers is modeled by a node. Table II summarizes the different nodes of the model.

TABLE II
H-FaTMA Architectural elements

Root	Node	Control	Arity	Atomic
Thing	Main	Main	0	No
	Backup	Backup	0	No
	Primary Thing	Thing	1	No
	Secondary Thing	Thing	1	No
	Sensor	T	1	Yes
	Actuator	A	1	Yes
	Reactive FT occurrence	RF	0	No
	Proacrtive FT occurrence	PF	0	No
Edge	Data Controller	DC	2	Yes
	Things Reactive Monitor	TR	3	Yes
	Decision Maker	DM	3	Yes
	Things Manager	TM	3	Yes
Fog	Data Processing	DP	3	Yes
	Things Proactive Monitor	TPM	2	Yes
	Fog-Things Supervisor	FTS	3	Yes
Cloud	Data Stroring	DS	2	Yes
	Application	APP	2	Yes
	Cloud-Things Supervisor	SCT	3	Yes
	Proactive Model Trainer	TPT	2	Yes

The formal definition of H-FaTMA infrastructure is as follows.

Definition: Formally, a bigraph FT modeling an instance of the H-FaTMA architecture is defined by $FT = Thing \parallel$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:52 UTC from IEEE Xplore. Restrictions apply.

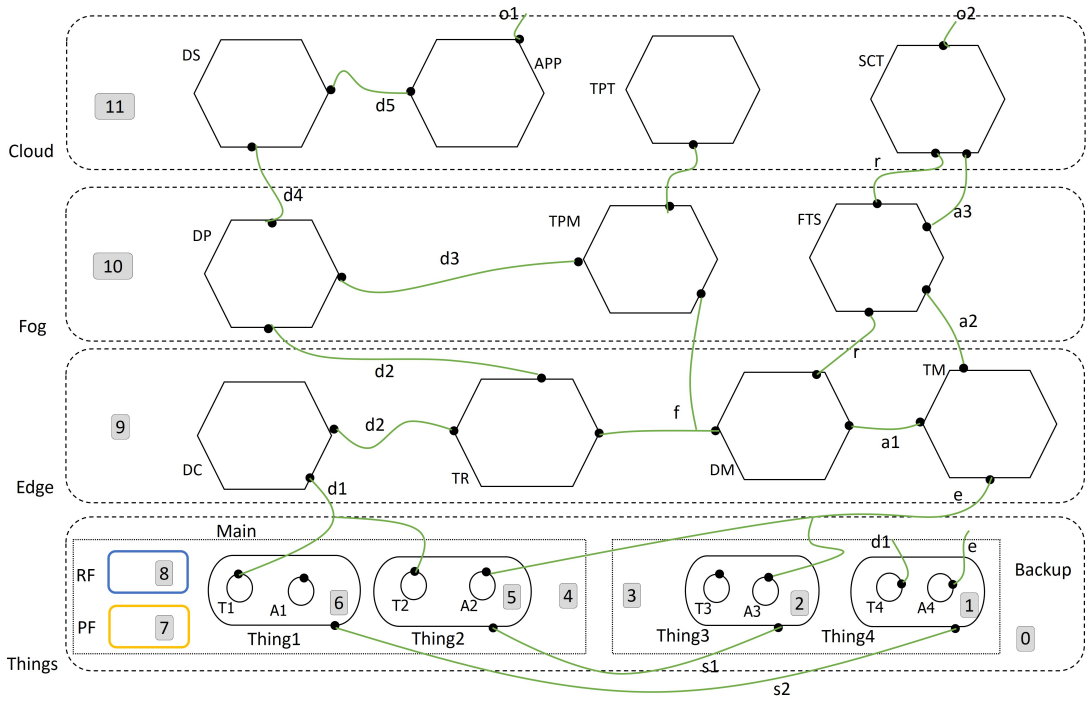


Fig. 3. H-FaTMA Bigraphical node types

$Edge \parallel Fog \parallel Cloud$ where $Thing, Edge, Fog$ and $Cloud$ are respectively the Things, Edge, Fog and Cloud layers bigraphs. $FT = (V_{FT}, E_{FT}, ctrl_{FT}, FT_P, FT_L) : I_{FT} \rightarrow J_{FT}$, where:

- $V_{FT} = V_{Thing} \uplus V_{Edge} \uplus V_{Fog} \uplus V_{Cloud}$ is a finite set nodes in H-FaTMA instance FT, given by the disjoint union of the sets of nodes of the architecture four layers .
- $E_{FT} = E_{Thing} \uplus E_{Edge} \uplus E_{Fog} \uplus E_{Cloud}$ is a finite set of edges consisting of the disjoint union of each if the architecture four layers sets of edges.
- $ctrl_{FT} = ctrl_{Thing} \uplus ctrl_{Edge} \uplus ctrl_{Fog} \uplus ctrl_{Cloud} : V_{FT} \rightarrow K_{FT}$, is a control map assigning to each node $v \in V_{FT}$ of the H-FaTMA bigraph (FT) a control node $k \in K_{FT}$ defining its identity. The signature $K_{FT} = K_{Thing} \uplus K_{Edge} \uplus K_{Fog} \uplus K_{Cloud}$ is the set of controls associated to FT.
- $FT_P = Thing_P \parallel Edge_P \parallel Fog_P \parallel Cloud_P$ is the place graph of FT given by the parallel product of Thing, Edge, Fog and Cloud place graphs.
- $FT_L = Thing_L \parallel Edge_L \parallel Fog_L \parallel Cloud_L$ is the link graph of FT given by the parallel product of Thing, Edge, Fog and Cloud link graphs.
- $I_{FT} = IFTThing \parallel I_{Edge} \parallel I_{Fog} \parallel I_{Cloud} = \langle m_{FT}, X_{FT} \rangle$, and $J_{FT} = JFTThing \parallel J_{Edge} \parallel J_{Fog} \parallel J_{Cloud} = \langle n_{FT}, Y_{FT} \rangle$, represent respectively the inner and outer interface of the bigraph FT. Where $(I, J) == (< n, \phi >, < m, 2 >)$.

Fig. 3 presents an illustrative instance of H-FaTMA with its four roots (region). In the Things root, we consider two connected things (Thing1 and Thing2 within the Main node) and two other things (Thing3 and Thing4) in the Backup node used for recovery. We assume that all Thing nodes are identical by having a sensor (T node) and an actuator (A node). Besides, the reactive FT and proactive FT occurrence is monitored respectively by RF and PF nodes. The other nodes represent an instances of H-FaTMA architectural elements with their interfaces (see Table II).

FT in its definition contains a set of nodes represented by V_{FT} that can interact together through links represented by the set E_{FT} in E_{FT} may be of the following types:

- Data-links=$\{D1, D2, D3, D4, D5\}$ where $D1, D2, D3, D4, D5$ corresponding respectively to "contextual data , gathered data ,processed data, saving data and data" H-FaTMA interfaces.
- Fault-Event-links= $\{f\}$.
- Action-links = $\{a, e\}$, where a is an action link and e is an executing-action link.
- Report-links= $\{r\}$.
- Switch-links = $\{s\}$.
- Outer-Events-links = $\{o\}$.

C. Specifying H-FaTMA FT mechanisms

In addition to their ability to model the H-FaTMA infrastructure, BRS allow the formal specification of an IoT system evolution by defining any expected behavior using the reaction rules. At this step, we proceed to propose a set of reaction rules "React" dealing with the Proactive and reactive FT. We define three categories of reaction rules; each category describes the

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:52 UTC from IEEE Xplore. Restrictions apply.

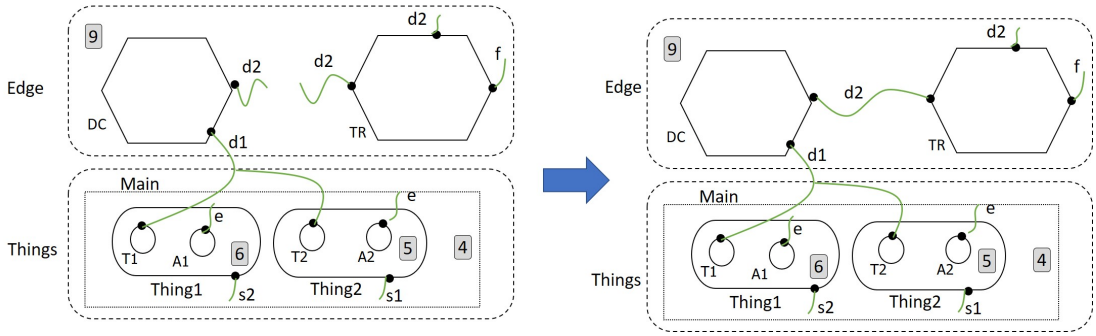


Fig. 4. Data flow reaction rule

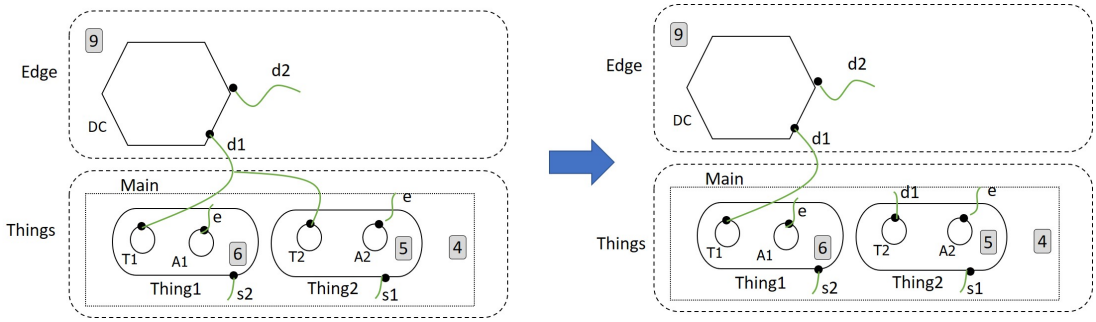


Fig. 5. Thing failure reaction rule

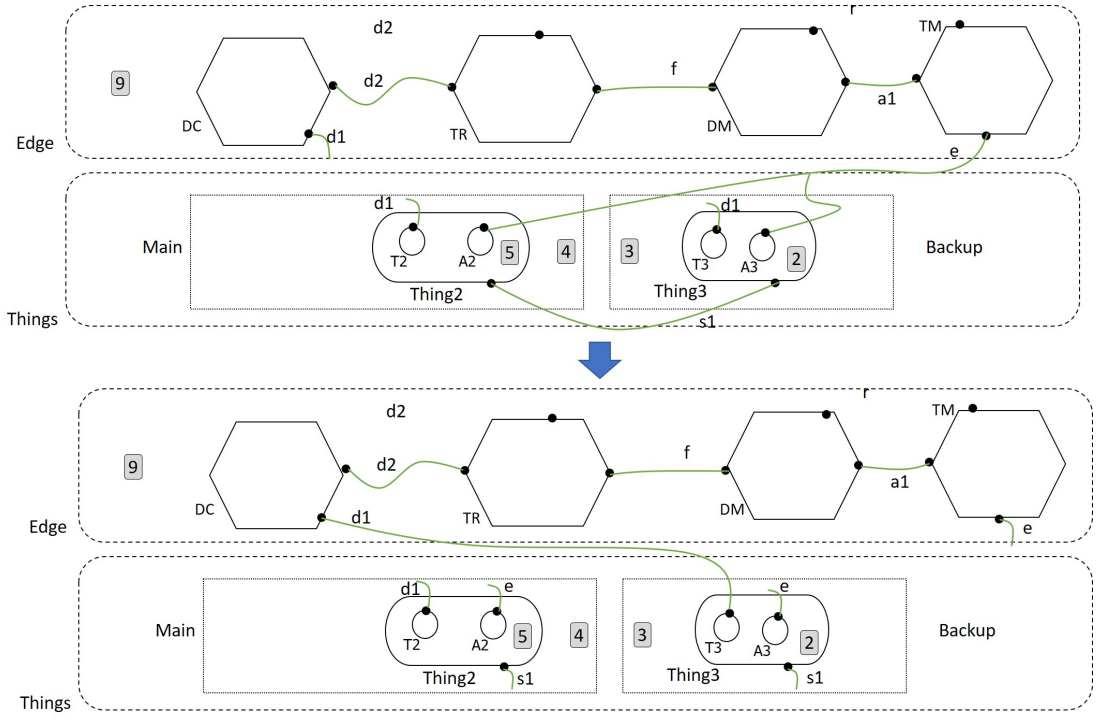


Fig. 6. Recovery reaction rule

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:52 UTC from IEEE Xplore. Restrictions apply.

semantics associated with a fault tolerance technique applied in the design of the architecture.

- **Meta-rules for data flow**: is a set of reaction rules that ensure the data transfer between the different bigraph nodes. Figure 4 illustrates data transmission from the *Data Controller* to the *Things Reactive Monitor* through the appearance of the *d2* link between the two nodes.
- **Failure Meta-rules:** these rules illustrate the appearance of device failure that can be by total crush of the object, sensor deviation or actuator not responding, etc. As an example, fig.5 describes a device breaking down.
- **Recovery Meta Rules**: are responsible for remediating the system according to the fault nature and severity. The figure 6 shows the replacement of a failed object by another. This rule is applied after a sequence of rules triggered when a failure is discovered. For each fault type, a set of reaction rules is defined in order to overcome system failure.

Table III groups together a set of H-FaTMA reaction rules divided by categories. Due to page restrictions, we focus on the transfer of data, the occurrence of failures and recovery behaviors (rules). There are other rules that cover the transfer of reports to the upper layers and the intervention of maintenance agents that are not presented in this paper.

V. Running H-FaTMA bigraphical model

The previously defined reaction rules can be used to simulate different H-FaTMA fault tolerance scenarios. Each scenario is an execution of a subset of reaction rules. In this section, we illustrate four possible scenarios. The initial state of the example is the following:
$Main.(Thing1_{s2}.(T1d1|A1)|Thing2_{s1}.(T2_{d1}|A2)|RF|PF)$
$|Backup.(Thing3_{s1}.(T3|A3)|Thing4_{s2}.(T4|A4))$ ‖
$DCd1|TR|DM|TM$ ‖ $DP|TPM|FTS$ ‖
$DS|APP|SCT|TPT$

a) Scenario 1: : is the save scenario, i.e. when no failure occurs. it is achieved by executing the following sequence of reaction rules: $R1 \rightarrow R2 \rightarrow R3$

b) Scenario 2: occurs when a connected object fails, i.e. it stops sending data to the Data Collector service. It is the simplest failure and the easiest to detect because it does not require any analysis. once this failure appeared a set of reaction rules are executed in order to remedy this problem, it is by this sequence: $R7 \rightarrow R8 \rightarrow R9 \rightarrow R10 \rightarrow R11$

c) Scenario 3: : concerns the failure that affects the safety of the data, i.e. the data received presents a flagrant contradiction. In this case; the reactive FT is the most appropriate and the following reaction rule set is executed to recover from the failure: $R12 \rightarrow R13 \rightarrow R14 \rightarrow R15 \rightarrow R16$.

VI. Conclusion

In this paper, we extended FaTMA to provide hybrid FT (proactive and reactive) support for IoT systems. In H-FaTMA, fault detection, data processing, recovery strategies and device management are separated and distributed all over the architectural layers. This will provide a decentralized fault control and management. Moreover, our architecture takes advantage of the microservices architectural style, Edge devices and Fog nodes to reinforce IoT systems dependability. These last will ensure more flexibility, low latency and week coupling between services. To validate our approach, we proposed BRS-based model for specifying H-FaTMA architectural and behavioral aspects. Indeed, our formal model allows the simulation and analysis of various fault tolerance scenarios.
Currently, we are working on the development of a concrete implementation of the H-FaTMA architecture and by applying it on real case studies. In addition, we will expand our BRS model to take into account LTL model checking and quality of service evaluation.

References

[1] Ghosh P., Eisele, S., Dubey, A., Metelko, M., Madari, I., Volgyesi, P., Karsai, G. (2020). Designing a decentralized fault-tolerant software framework for smart grids and its applications. Journal of Systems Architecture, 109, 101759.

[2] Sahar Smaali, Benbessem Riadh, Touati Hatem Mohamed Nazim (2022) "A Fault Tolerance and Recovery Formal Model for IoT Systems", International Journal of Organizational and Collective Intelligence, IGI-global, vol. 12, no 2, p. 1-24.

[3] R. Milner (2008). Bigraphs and their algebra. Electronic Notes in Theoretical Computer Science, 209, 5–19.

[4] Milner, R. (2009). The Space and Motion of Communicating Agents, Cambridge university Press, Cambridge.

[5] M. Z. Hasan, Al-Turjman, F. Optimizing multipath routing with guaranteed fault tolerance in Internet of Things. IEEE Sensors Journal, 17(19), 6463-6473. 2017.

[6] S. Sivakumar, P. Vivekanandan. Efficient fault-tolerant routing in IoT wireless sensor networks based on path graph flow modeling with Marchenko–Pastur distribution (EFT-PMD). Wireless networks, 26, 4543-4555,2020.

[7] K. aiswal, V. Anand, "FAGWO-H: A hybrid method towards fault-tolerant cluster-based routing in wireless sensor network for IoT applications." J Supercomput 78, 11195–11227, 2022. https://doi.org/10.1007/s11227-022-04333-6

[8] N. Cheraghlou, A. Khadem-Zadeh and M. Haghparast . "A new hybrid fault tolerance approach for Internet of Things ". Electronics, 8(5), 518, 2019.

[9] P. H. Su, C.-S. Shih, J. Y.-J. Hsu, K.-J. Lin, Y.-C. Wang, "Decentralized fault tolerance mechanism for intelligent IoTM2M middleware". IEEE world forum on internet of Things (wf-iot) (pp. 45–50), 2014.

[10] P. K. Choubey, S. Pateria, A. Saxena, Vaisakh Punnekkattu Chirayil SB, K. K. Jha and Sharana Basaiah PM, "Power efficient, bandwidth optimized and fault tolerant sensor management for IOT in Smart Home," 2015 IEEE International Advance Computing Conference (IACC), 2015, pp. 366-370, doi: 10.1109/IADCC.2015.7154732.

[11] A. Power and G. Kotonya, "A Microservices Architecture for Reactive and Proactive Fault Tolerance in IoT Systems," 2018 IEEE 19th International Symposium on "A World of Wireless, Mobile and Multimedia Networks" (WoWMoM), 2018, pp. 588-599, doi: 10.1109/WoWMoM.2018.8449789.

[12] A. Power and G. Kotonya, "Providing Fault Tolerance via Complex Event Processing and Machine Learning for IoT Systems". In Proceedings of the 9th International Conference on the Internet of Things (IoT 2019). Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3365871.3365872

[13] M. Melo and G. Aquino, "FaTEMa: A Framework for Multi-Layer Fault Tolerance in IoT Systems", Sensors, vol. 21, no. 21, p. 7181, 2021. Available: 10.3390/s21217181.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:52 UTC from IEEE Xplore. Restrictions apply.

TABLE III
H-FATMA REACTION RULES

Rules for data flow	
R1	$Main.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1} \mid TR \mid d_2 \rightarrow$ $Main.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid d_2$
R2	$Main.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid d_2 \parallel DP \mid d_3 \rightarrow$ $Main.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid d_2 \parallel DP_{d2} \mid d_3$
R3	$Main.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid d_2 \parallel DP_{d2} \mid TPM \mid d_3 \parallel DS \mid d_3 \rightarrow$ $Main.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid d_2 \parallel DP_{d2\,d4\,d3} \mid TPM_{d3} \mid d_3 \parallel DS_{d4} \mid d_3$
Failure rules	
R4	$Main.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1} \mid d_2 \rightarrow$ $Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1} \mid d_2$
R5	$Main.(Thing1_{s2}.(T1_{d1} \mid A1) \mid RF.d_3 \mid d_0)) \mid d_1 \parallel DC_{d1} \mid d_2 \rightarrow$ $Main.\ RF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1} \mid d_2$
R6	$Main.(Thing1_{s2}.(T1_{d1} \mid A1) \mid PF.d_3 \mid d_0)) \mid d_1 \parallel DC_{d1} \mid d_2 \rightarrow$ $Main.\ PF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1} \mid d_2$
Recovery Rules	
R7	$Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1} \mid TR \mid d_2 \rightarrow$ $Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid d_2$
R8	$Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid DM \mid d_2 \rightarrow$ $Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_f \mid d_2$
R9	$Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_f \mid TM_e \rightarrow$ $Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_{f\,a1} \mid TM_{e\,a1}$
R10	$Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4 \mid A4) \mid d_1) \mid d_2 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_{f\,a1} \mid TM_{e\,a1} \rightarrow$ $Main.(Thing1_{s2}.(T1 \mid A1_e) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4 \mid A4_e) \mid d_1) \mid d_2 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_{f\,a1} \mid TM_{e\,a1}$
R11	$Main.(Thing1_{s2}.(T1 \mid A1_e) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4 \mid A4_e) \mid d_1) \mid d_2 \parallel DC_{d1} \mid d_3 \rightarrow$ $Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4_{d1} \mid A4) \mid d_1) \mid d_2 \parallel DC_{d1} \mid d_3$
R12	$Main.\ RF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1} \mid TR \mid d_2 \rightarrow$ $Main.\ RF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid d_2$
R13	$Main.\ RF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid DM \mid d_2 \rightarrow$ $Main.\ RF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_f \mid d_2$
R14	$Main.\ RF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_f \mid TM \mid d_2 \rightarrow$ $Main.\ RF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_{f\,a1} \mid TM_{a1} \mid d_2$
R15	$Main.\ RF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid Backup.(Thing4_{s2}.(T4 \mid A4) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_{f\,a1} \mid TM_{a1} \mid d_2 \rightarrow$ $Main.(Thing1_{s2}.(T1_{d1} \mid A1_e) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4 \mid A4_e) \mid d_1) \mid d_2 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_{f\,a1} \mid TM_{a1\,e} \mid d_3$
R16	$Main.(Thing1_{s2}.(T1_{d1} \mid A1_e) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4 \mid A4_e) \mid d_1) \mid d_2 \parallel DC_{d1\,d2} \mid d_3 \rightarrow$ $Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4_{d1} \mid A4) \mid d_1) \mid d_2 \parallel DC_{d1\,d2} \mid \mid d_3$
R17	$Main.\ PF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1} \mid TR \mid d_2 \rightarrow$ $Main.\ PF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid d_2$
R18	$Main.\ PF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2} \mid DM \mid d_2 \rightarrow$ $Main.\ PF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_f \mid d_2$
R19	$Main.\ PF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_f \mid TM \mid d_2 \rightarrow$ $Main.\ PF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_{f\,a1} \mid TM_{a1} \mid d_2$
R20	$Main.\ PF.(Thing1_{s2}.(T1_{d1} \mid A1) \mid Backup.(Thing4_{s2}.(T4 \mid A4) \mid d_0) \mid d_1 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_{f\,a1} \mid TM_{a1} \mid d_2 \rightarrow$ $Main.(Thing1_{s2}.(T1_{d1} \mid A1_e) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4 \mid A4_e) \mid d_1) \mid d_2 \parallel DC_{d1\,d2} \mid TR_{d2\,f} \mid DM_{f\,a1} \mid TM_{a1\,e} \mid d_3$
R21	$Main.(Thing1_{s2}.(T1_{d1} \mid A1_e) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4 \mid A4_e) \mid d_1) \mid d_2 \parallel DC_{d1\,d2} \mid d_3 \rightarrow$ $Main.(Thing1_{s2}.(T1 \mid A1) \mid d_0)) \mid Backup.(Thing4_{s2}.(T4_{d1} \mid A4) \mid d_1) \mid d_2 \parallel DC_{d1\,d2} \mid \mid d_3$
R22	$Main.\ PF.(Thing1s2.(T1d1 \mid A1) \mid Backup.(Thing4s2.(T4 \mid A4) \mid d0) \mid d1 \parallel DCd1\ d2 \mid TMa1 \mid d2 \rightarrow$ $Main.(Thing1s2.(T1d1 \mid A1e) \mid d0)) \mid Backup.(Thing4s2.(T4 \mid A4e) \mid d1) \mid d2 \parallel DCd1\ d2 \mid TMa1\ e \mid d3$
R23	$Main.(Thing1s2.(T1d1 \mid A1e) \mid d0)) \mid Backup.(Thing4s2.(T4 \mid A4e) \mid d1) \mid d2 \parallel DCd1\ d2 \mid d3 \rightarrow$ $Main.(Thing1s2.(T1 \mid A1) \mid d0)) \mid Backup.(Thing4s2.(T4d1 \mid A4) \mid d1) \mid d2 \parallel DCd1\ d2 \mid d3$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:52 UTC from IEEE Xplore. Restrictions apply.